The Cost of Deadlines: Empirical Dynamics of Post-Quantum Migration in Blockchain Systems

Using Bitcoin's Proposed BIP-360 / BIP-361 Migration Policy as a Controlled Case Study

Senior Project Proposal (SProj)

Abyaan Shivjani, 27100110 & Research Partner

Computer Science, LUMS

CONTENTS

1. Background

1.1 Why Blockchains Depend on Digital Signatures

1.2 The Quantum Threat

1.3 The Response: Post-Quantum Cryptography

1.4 Bitcoin's Concrete Migration Plan

1.5 The Cost Nobody Has Measured Yet

2. Problem Statement

2.1 What the Existing Literature Already Covers

2.2 The Specific Gap

2.3 Research Question

3. Research Framing

3.1 A Testbed, Not a Reimplementation

3.2 A Narrow, Defensible Novelty Claim

3.3 What This Study Does and Does Not Claim About Bitcoin

4. System Design — What Will Be Built

4.1 Architecture Overview

4.2 Blockchain Core

4.3 Cryptographic Interface

4.4 Transaction and UTXO Model

4.5 Migration and Phase Engine

4.6 Metrics and Experiment Runner

4.7 Python Analysis Pipeline

5. Experimental Design — What Will Be Researched

5.1 Core Experiments

5.2 Sensitivity Experiments

5.3 Reproducibility

6. Metrics and Calculations — What Will Be Measured and Computed

6.1 Three Primary Metrics

6.2 Derived Metrics

6.3 Statistical Methodology

6.4 Handling Measurement Noise

6.5 Strength of Evidence Across Metrics

7. Scope Boundaries — What Will Not Be Done

8. Project Tiers and Timeline

8.1 Scope Hierarchy

8.2 Semester 1 — Build the Core System

8.3 Semester 2 — Run the Study and Write the Paper

9. Feasibility

10. Expected Contribution and Conclusion

1. Background

1.1 Why Blockchains Depend on Digital Signatures

Every blockchain, Bitcoin included, is fundamentally a decentralized ledger of who is allowed to move what. The mechanism that enforces this is the digital signature: a piece of mathematics that lets someone prove they authorized a transaction without ever revealing the private information that makes that proof possible. Every transaction on the network carries one of these signatures, every other participant checks it before accepting the transaction, and the entire idea of ownership on the network rests on the assumption that a signature cannot be forged by anyone other than the person who holds the corresponding private key.

Bitcoin's signatures are built on elliptic curve cryptography, a curve called secp256k1. Elliptic curve signatures are small, fast to verify, and, as far as any computer that exists today is concerned, computationally infeasible to break. A signature is typically only 64 to 72 bytes, which is a large part of why Bitcoin has been able to fit thousands of transactions into a single one-megabyte block.

1.2 The Quantum Threat

The security of elliptic curve cryptography does not rest on the problem being unsolvable in principle. It rests on the problem being computationally impractical for any classical computer to solve in a useful amount of time. A cryptographically relevant quantum computer (CRQC) changes that calculation. Using Shor’s algorithm, developed by Peter Shor in 1994, a quantum computer with enough stable, error-corrected qubits would be able to derive a private key from its corresponding public key in a feasible amount of time, which is something that would take a classical computer longer than the age of the universe.

No such computer exists today, and credible estimates place the arrival of one anywhere from the late 2030s to well past 2050, with meaningful uncertainty on both ends. But two things make this a present-tense engineering problem rather than a distant hypothetical. First, cryptographic migrations at internet scale take years to design and even longer to roll out safely, so waiting until the threat is imminent is itself a risk. Second, an adversary can record encrypted or signed data today and decrypt or forge it retroactively once a CRQC exists which is a strategy known as "harvest now, decrypt later." For a public, permanent ledger like Bitcoin, where every public key that has ever spent a transaction is visible forever, this second point matters enormously: a big fraction of all Bitcoin in existence sits in addresses whose public keys are already exposed on-chain, waiting.

1.3 The Response: Post-Quantum Cryptography

In response, cryptographers have spent the last decade designing signature and encryption schemes believed to resist attacks from both classical and quantum computers. In 2024, the U.S. National Institute of Standards and Technology (NIST) finalized the first set of these as official standards, including a lattice-based signature scheme called ML-DSA (standardized as FIPS 204, also known as Dilithium). These are built on entirely different mathematical foundations, and that difference has a direct, unavoidable cost: post-quantum signatures are dramatically larger and more computationally expensive to verify than the elliptic curve signatures they are meant to replace. A typical ML-DSA signature runs from roughly 2,400 to over 4,600 bytes depending on the security level chosen which can be about forty to seventy times the size of the ECDSA signature it would replace. Some alternative post-quantum schemes are even larger, running into the tens of kilobytes.

1.4 Bitcoin's Concrete Migration Plan

Bitcoin's developers are not proposing to switch cryptographic schemes overnight; a change of that scale would break consensus. Instead, two connected proposals currently moving through Bitcoin's improvement-proposal process lay out a gradual, multi-year transition.

The first, BIP-360, introduces a new type of address known as P2MR (Pay to Merkle Root), designed to be quantum-resistant. This gives the network a destination to migrate toward, sitting alongside Bitcoin's existing address types rather than replacing them.

The second, BIP-361, is a retirement plan for the old, quantum-vulnerable address types, and it is the proposal this project is built around. It defines a three-phase sunset:

- Phase A: Roughly three years (approximately 160,000 blocks) after soft fork activation, the network stops accepting new payments into legacy, quantum-vulnerable address types. Funds already held in those addresses remain spendable exactly as before.
- Phase B: Around two years after Phase A (five years after activation in total), the network stops verifying legacy ECDSA and Schnorr signatures. Any coins still sitting in a legacy address at this point become frozen: technically present on the ledger, but no longer spendable, because the network will no longer accept the signature required to move them.
- Phase C: A proposed, not-yet-designed recovery mechanism that would eventually let the rightful owners of frozen coins reclaim them by proving ownership through some other means, without exposing them to the same theft risk. Because this phase has no finalized design, it falls outside the scope of what this project measures.

Current estimates put roughly a third of all circulating bitcoin in addresses that BIP-361 would eventually affect. A meaningful share of that is coins whose owners have lost their keys or are otherwise gone, meaning no deadline, however generous, will ever bring them across voluntarily.

1.5 The Cost Nobody Has Measured Yet

Post-quantum signatures are both necessary and are forty to seventy times larger, and two to five times more costly to verify, than what they replace. BIP-361 does not let the network absorb that cost gradually and indefinitely; it imposes two hard, dated deadlines after which the cost becomes non-negotiable for anyone still holding legacy funds. What happens to the network's performance, capacity, and accumulated data load as it moves through BIP 361, phase by phase, is the question this project sets out to answer.

2. Problem Statement

2.1 What the Existing Literature Already Covers

The general costs of post-quantum cryptography are well studied. Large systematic literature reviews already catalogue signature and key sizes, verification costs, and block-capacity implications across dozens of proposed schemes and blockchain designs. That work is mature, and this project does not attempt to add to it. The byte-level and CPU-level costs of switching from ECDSA to ML-DSA are treated here as established background, not as something to be discovered.

2.2 The Specific Gap

What is missing is much narrower and much more concrete: nobody has taken a multi-phase migration policy such as BIP-361 and measured what actually happens to a network's block capacity, validation latency, and accumulated data overhead as it passes through that policy's specific enforcement boundaries. The existing literature answers "how much more expensive is a post-quantum signature." It does not answer "what does a network experience the moment a hard migration deadline arrives, compared to what it would have experienced if

migration had simply continued on its own." BIP-360 and BIP-361 are recent proposals: BIP-360 was only merged into Bitcoin's official proposal repository in February 2026, and BIP-361 followed as a draft in April 2026, so the research literature has had essentially no time to catch up to a plan this specific.

2.3 Research Question

> *How does the timing and enforcement of post-quantum signature migration affect blockchain block capacity, validation latency, and accumulated data overhead — and how do these effects differ between gradual, voluntary adoption and migration constrained by a hard, protocol-enforced deadline?*

A second question sits underneath the first and gives the project its sharpest potential result: can two networks that end up in exactly the same final state: **fully migrated to post-quantum signatures**, and nonetheless experience very different peak stress and very different cumulative infrastructure cost along the way, purely as a function of how that migration was paced and enforced? If so, the path a migration takes matters just as much as its destination, which has direct, practical implications for how a real deadline-based policy like BIP-361 should be designed.

3. Research Framing

3.1 A Testbed, Not a Reimplementation

This project's testbed is no longer a from-scratch stand-in for Bitcoin: it is built directly on btq-core, an open-source fork of Bitcoin Core that already implements BIP-360 (P2MR addresses and ML-DSA signature opcodes) on real, consensus-critical C++ code. Running actual Bitcoin nodes, rather than an independent simplified simulator, was a scope change requested directly by the project's advisor, and it changes what "not a reimplementation" means for this project without abandoning the principle behind it. We do not re-implement Bitcoin's cryptography, P2P stack, or consensus engine ourselves; we consume btq-core's existing, real implementation of those layers. The one piece of consensus logic this project does add by hand is BIP-361's phased sunset rule (Section 4.5), patched directly into btq-core's validation code, because no existing implementation of that policy exists anywhere to build on. Every other simplification retained from the original design, principally a controlled, deterministic adoption trajectory rather than an open-ended behavioral model, exists for the same reason as before: to remove variables that are irrelevant to the research question, so the variable that matters, migration pacing and enforcement timing, can be measured cleanly on top of a real node.

3.2 A Narrow, Defensible Novelty Claim

This project does not claim that post-quantum blockchain overhead is an unexplored topic, it is not. The claim is narrower and more specific: this is a phase-aware, temporal study of migration dynamics under a dated enforcement policy, centered on the difference between

organic and deadline-driven migration. In contrast to a sweeping gap, the empirical gap is not only a more accurate statement about what the contribution entails but is also easier to defend in a literature review.

3.3 What This Study Does and Does Not Claim About Bitcoin

It is important to state plainly what kind of evidence this project produces, because it is easy to misread a simulator built around a real policy as a prediction about that real system. This project does not claim to predict Bitcoin's actual future latency, block size, or fee behavior under BIP-361. Instead, the testbed is a simplified abstraction, not a forecasting model, and no numeric result from it should be read as "Bitcoin will experience X."

What the project does produce is causal, relative evidence about migration dynamics: “under a controlled abstraction of BIP-361's policy, does enforcing a hard deadline change peak stress and cumulative cost compared to the same migration left to happen gradually, and is that difference robust to reasonable changes in migration speed and adoption assumptions”. That is a question about the structure of deadline-driven migration in general, illustrated through Bitcoin's specific, real proposal and not a quantitative prediction about Bitcoin itself. Keeping this distinction explicit is what allows the testbed's simplifications (Section 3.1) to be a strength rather than a liability: the simulator is not trying to be realistic enough to forecast Bitcoin, only controlled enough to isolate the effect of enforcement timing.

4. System Design: What Will Be Built

The system is organized into six modules, each with a single clear responsibility. This is a deliberate reduction from a more ambitious original design: every module here exists because the research question depends on it, and nothing is included merely because it would be technically interesting.

4.1 Architecture Overview

- Blockchain core: btq-core, a real Bitcoin Core fork, running as an actual consensus-validating node — not a custom, in-process consensus loop.
- Cryptographic interface: btq-core's existing, real ECDSA and ML-DSA (BIP-360 P2MR) implementations, consumed as-is rather than built by this project.
- Transaction / UTXO model: The real UTXO set maintained by each btq-core node, queried over RPC to make block bloat and capacity measurable and reproducible.
- Migration and phase engine: Generates the adoption trajectory and, through our BIP-361 patch inside btq-core, enforces real Phase A/B consensus rules — no longer simulated.
- Metrics and experiment runner: Polls every node over RPC each round and automates the experiment matrix across the 6-node mesh.
- Python analysis pipeline: Turns logs into the figures and statistics reported in the paper.

- Infrastructure and deployment: Six free-tier Oracle Cloud VMs, networked as a private regtest mesh, hosting the node cluster (Section 4.8).

4.2 Blockchain Core

Six btq-core nodes run as real, independent processes, one per VM (Section 4.8), communicating over genuine P2P network sockets on a private regtest mesh rather than in-process channels. Real networking is now part of the design rather than something explicitly excluded, since the advisor's brief calls for actual Bitcoin nodes; the mesh is kept private and single-region specifically to keep network-layer noise manageable while still being a real, physically distributed system. Block production follows a scripted round-robin model: an orchestration script rotates which node calls the RPC method to generate the next block, and the block propagates to the other five nodes over real P2P, where it is independently validated, including against our BIP-361 patch (Section 4.5). This consensus loop is btq-core's own, unmodified beyond our patch; it is treated purely as an experimental substrate and is not itself a research contribution.

4.3 Cryptographic Interface

No signing interface is built by this project. btq-core already ships both backends as real, consensus-validated code: classical ECDSA/Schnorr signatures over secp256k1, matching Bitcoin's real cryptography, and post-quantum ML-DSA signatures (FIPS 204) exposed through BIP-360's P2MR output type and its dedicated signature opcode. Both are real, correct implementations already exercised on btq-core's public testnet; nothing here is simulated or approximated, and none of it needs to be written by this project. The project deliberately restricts itself to the single ML-DSA parameter set btq-core ships as its core experiment rather than expanding into a broad comparison across multiple post-quantum schemes; a second parameter set, if btq-core exposes one, is treated strictly as an optional Tier 3 extension, not a requirement.

4.4 Transaction and UTXO Model

The transaction format is no longer one this project defines: it is btq-core's real, on-chain format, including its legacy input/output types and its BIP-360 P2MR witness structure. Because that format is fixed by the software under test rather than by our own modeling choices, every block-bloat result traces back to a real, reproducible transaction format rather than an assumption we made ourselves.

Each unspent transaction output (UTXO) in each node's real UTXO set carries an address-type tag (legacy or post-quantum) inherent to its script type, and effectively becomes frozen once our Phase B patch rejects any transaction that would spend it. Rather than modeling a frozen flag ourselves, we read the real UTXO set directly via gettxoutsetinfo and our own address-type classification, which keeps the result faithful to what BIP-361 actually proposes and produces a genuine result to report: how many UTXOs, and what proportion of value, end up permanently frozen under each migration scenario.

4.5 Migration and Phase Engine

An adoption function determines, at each point in simulated time, what fraction of newly generated transactions use post-quantum versus legacy addresses. The Tier 1 function used is a fixed logistic curve, chosen because it is the standard shape for describing voluntary technology adoption: slow at first, accelerating through a middle period, then leveling off. Its parameters (ceiling, steepness, midpoint) are fixed once per experiment configuration rather than fit dynamically, since the goal is to generate a controlled, known adoption behavior that keeps the E1-versus-E2 comparison in Section 5.1 causally clean: because the same fixed trajectory drives both conditions, any difference between them can only be attributed to enforcement. A richer, behavior-driven adoption model, in which simulated holders react to how close the network is to a freeze deadline, is treated as an optional Tier 3 extension (Section 8.1) rather than a Tier 1 requirement, precisely because it would make the trajectory itself a function of enforcement and complicate that clean isolation.

Two enforcement rules are patched directly into btq-core's consensus validation code, each gated by a configurable block-height threshold that stands in for BIP-361's real Phase A and Phase B triggers: Phase A, added inside CheckTxInputs, rejects any transaction that would create a new output to a legacy address; Phase B, added inside ConnectBlock / CheckInputScripts, rejects verification of any legacy-signed transaction outright, which leaves its UTXOs unspendable and therefore effectively frozen. Because the mesh runs in regtest rather than on real proof-of-work, blocks are generated on demand rather than mined, so a single, explicitly documented time-compression constant maps BIP-361's real-world timings (roughly 160,000 blocks for Phase A, roughly five years total for Phase B) onto the number of regtest blocks generated per run, and this mapping is reported verbatim in the paper rather than hidden inside configuration files. The specific btq-core commit or tag the patch is built against is pinned and recorded for the same reason (Section 5.3), since btq-core is an actively developed, fast-moving fork.

4.6 Metrics and Experiment Runner

Every consensus round is captured over each node's RPC interface (getblock, getrawmempool, gettxoutsetinfo), supplemented by our own debug-log lines around the Phase A/B patch, with, at minimum: block size in bytes, average signature size, transaction count broken down by legacy versus post-quantum, frozen UTXO count, end-to-end consensus round latency across the 6-node mesh, and the identity of the round's block-generating node. Wall-clock timing is the default and sufficient measurement tier; more granular CPU-cycle profiling through a tool such as perf is treated as an optional addition, not a requirement. Output is logged to CSV, which is sufficient for a dataset of this size; a local SQLite database is an optional convenience if querying across many experiment runs becomes difficult. A Python orchestration script drives the full experiment matrix (described in Section 5) unattended across all six VMs, triggering block generation on a rotating leader and polling every node, so that once configured, an entire set of runs can execute as a single automated batch.

4.7 Python Analysis Pipeline

Once raw logs exist, analysis moves to Python. pandas loads and merges results across the experiment matrix; scipy is used to confirm the observed adoption trajectory matches its configured logistic shape; matplotlib produces the final figures: block capacity, validation latency, and cumulative overhead over simulated time, with Phase A and Phase B boundaries clearly marked, alongside the derived metrics defined in Section 6.

4.8 Infrastructure and Deployment

The node cluster runs on six Oracle Cloud Always Free-tier VMs, provisioned in a single home region on a private Virtual Cloud Network so the mesh is fast and controllable without being exposed to the public internet. Free-tier compute is limited and split across two shapes: two AMD micro instances (1/8 OCPU, 1 GB RAM each) and an Ampere A1 ARM pool (current allocation confirmed against the live account quota before deployment, since Oracle has changed this limit before). Because the two shapes differ substantially in power and architecture, they are not treated as interchangeable in the experiment design:

- The Ampere A1 (arm64) instances run as the full validating btq-core nodes used in the E0/E1/E2 comparison, since they carry the bulk of the free-tier compute budget.
- The AMD micro instances, given their 1 GB RAM ceiling, are used as lightweight roles (RPC/transaction-injection clients or the orchestration controller) rather than counted as validating consensus nodes, to avoid confounding latency results with hardware that cannot represent the other five nodes fairly.
- btq-core's build is verified on the target architecture (arm64) before any patch work begins; ML-DSA's cryptographic backend is checked specifically, since post-quantum libraries sometimes carry x86-only optimized code paths that would need a portable fallback on ARM.

If btq-core does not build or run acceptably within this free-tier budget, the documented fallback is to keep the original in-process Go blockchain design (Section 4.2's predecessor) but deploy it as genuinely separate networked processes across the same VMs, using real sockets instead of in-process channels. This still satisfies real, networked nodes running on VMs at substantially lower engineering risk, and is treated as an explicit Tier 1 alternative, not a failure state, should the primary path prove infeasible early in Semester 1.

5. Experimental Design: What Will Be Researched

5.1 Core Experiments

The heart of the project is a controlled, three-condition comparison. The gradual and enforced conditions share the exact same underlying adoption trajectory, the only difference between them is whether the simulated Phase A/B enforcement rules are switched on. This is what makes the comparison causally interpretable: if the two conditions used different adoption curves, any observed difference in outcomes could be blamed on the curve shape rather than on enforcement itself. By holding the trajectory fixed and toggling only enforcement, the experiment isolates enforcement as the treatment variable.

ID	Condition	Adoption Trajectory	Enforcement
E0	Classical baseline	0% post-quantum	None
E1	Gradual migration	Fixed logistic trajectory A(t)	None
E2	Enforced migration	Same trajectory A(t)	Phase A + Phase B

> *E1 and E2 are not simply two scenarios to compare; they form a controlled experiment in which protocol enforcement is the treatment variable, and the shared adoption trajectory is the control that makes the comparison meaningful.*

5.2 Sensitivity Experiments

To confirm that the core result is not an artifact of one arbitrarily chosen configuration, three further axes are tested, each varied one at a time against the fixed baseline configuration, never combined into a full factorial grid. A full cross of every axis considered during planning would have produced several hundred configurations before repetitions; varying one factor at a time instead produces a small, interpretable set of additional runs while still answering the question of which factors the result depends on.

ID	Variable	Levels	Priority
S1	Migration speed (adoption curve steepness)	Slow / Medium / Fast	Required
S2	Adoption curve shape	Logistic / Linear	Required
S3	System capacity (block/transaction limits)	Low / Medium / High	Recommended, not required

Two scenario types considered during planning were deliberately folded into this simpler structure rather than kept as separate named conditions. "Early-adopter" and "late-adopter" behavior is fully captured by varying the adoption curve's steepness and midpoint under S1, rather than being treated as additional standalone scenarios. This keeps the experimental design centered on one clear question (gradual versus enforced) with one clear sensitivity axis around it (how fast), rather than fragmenting into many loosely related scenarios.

5.3 Reproducibility

Every experiment run records its complete configuration: random seed, cryptographic scheme and parameter set, node count and VM shape, adoption curve type and parameters, phase-boundary block heights, system capacity settings, the exact Git commit of our BIP-361 patch, and the exact upstream btq-core commit or tag it was built against, since btq-core is an actively developed fork and building against a moving target would make runs incomparable. Every figure in the final paper is therefore traceable back to the specific configuration and code version that generated it.

6. Metrics and Calculations: What Will Be Measured and Computed

6.1 Three Primary Metrics

- Block capacity: Transactions successfully included per block.
- Validation and consensus performance: Signature verification time and end-to-end consensus round latency, measured and reported separately (see 6.4).
- Cumulative data overhead: The additional bytes the network processes as a direct result of migration, accumulated over the whole run.

These three are the paper's headline results. Every other quantity described below is a derived statistic computed from these three, keeping the paper anchored to three well-understood measurements rather than a long, harder-to-interpret list of headline numbers.

6.2 Derived Metrics

Latency amplification, for a given scenario s, relative to the classical baseline:

$$LA_s = mean(latency_s) / mean(latency_{baseline})$$

Capacity degradation, where C is transactions successfully included per block:

$$CD_s = 1 - (mean(C_s) / mean(C_{baseline}))$$

Phase shock measures how sharply a metric M changes in the window immediately after a phase boundary t_p, compared to the window immediately before it, using a fixed, pre-registered window width w:

$$PS = (M[t_p, t_p+w] - M[t_p-w, t_p]) / M[t_p-w, t_p]$$

The window width w is fixed before any experiment is run, using the following rule rather than an arbitrary round count: w is chosen as at most 25% of the number of simulated rounds between Phase A and Phase B. This guarantees the two phase-shock measurements, one for each boundary, never overlap and contaminate each other, which would otherwise be a real risk given how close together the two phases are once time is compressed.

Cumulative migration overhead requires one precise definition before it can be trusted. Raw block size at a given round is not automatically "extra bytes caused by migration": block capacity itself can change under migration, since a signature-heavy block may hold fewer transactions while still being larger overall. Comparing round-by-round block size directly, without accounting for this, risks conflating pure cryptographic overhead with a separate capacity effect.

To avoid that confound, every scenario is run against the same fixed total transaction demand D — an identical number of transactions that must be processed over the course of the run, generated identically across all conditions regardless of how many fit per block. Cumulative

migration overhead is then defined as the difference in total bytes required to process that same fixed demand, comparing a scenario to the classical baseline:

$$CMO_bytes = TotalBytes_scenario(D) - TotalBytes_baseline(D)$$

Defined this way, the metric is precisely a cumulative byte overhead relative to the classical baseline under equivalent transaction demand and not simply a sum of whatever block sizes happen to occur, which keeps the number defensible against direct questioning about what it actually measures.

Deadline amplification compares peak stress under enforced migration to peak stress under gradual migration, computed separately per metric rather than as one composite score:

$$DA_L = PeakLatency_enforced / PeakLatency_gradual$$

$$DA_B = PeakBlockSize_enforced / PeakBlockSize_gradual$$

This is the project's most directly novel measurement: it quantifies, in a single ratio per metric, how much worse the worst moment of a deadline-enforced migration is compared to the worst moment of the same migration left to happen gradually.

6.3 Statistical Methodology

Each experiment configuration is run a minimum of five independent times. For every primary metric, results are reported as the mean, standard deviation, and 95% confidence interval across those runs. Where a direct comparison between two conditions is useful (E1 versus E2). A Mann-Whitney U test is used as an additional, non-parametric check, since it does not assume the underlying data is normally distributed.

Five repetitions is a modest sample size, and the paper treats any resulting p-value accordingly: statistical significance is interpreted alongside effect size and confidence intervals, never reported as a standalone, self-sufficient claim of proof. If the simulator proves to be highly deterministic in practice, the necessity of five repetitions per configuration for every metric will be revisited rather than assumed to be fixed in advance.

6.4 Handling Measurement Noise

Running multiple validator processes on a single shared machine, without any CPU isolation, introduces operating-system scheduling noise into any wall-clock latency measurement. Rather than reintroducing full CPU-quota isolation through Linux cgroups, this noise is managed more simply: background workload on the measurement machine is minimized, validator processes are pinned to fixed CPU cores using a lightweight tool such as taskset where practical, and enough repetitions are run to report an honest, appropriately wide confidence interval rather than a falsely precise single number.

A further methodological distinction is maintained throughout: raw cryptographic verification time (how long it takes to check one signature, in isolation) is benchmarked and reported separately from end-to-end consensus round latency (how long an entire round takes, including scheduling and coordination overhead). The first is a clean, low-noise measurement; the second necessarily carries some orchestration noise. Reporting them

separately, rather than only the harder-to-interpret combined number, makes the final results substantially easier to defend.

6.5 Strength of Evidence Across Metrics

Not all four primary and derived measurements carry equal evidentiary weight, and the paper is written to reflect that explicitly rather than treat every number as equally conclusive. Ranked from cleanest to noisiest:

- Signature verification cost: The cleanest measurement available. Verifying N signatures in isolation involves minimal environmental noise and can be benchmarked directly against the cryptographic library's own documented performance.
- Block capacity / transaction throughput: Very clean, since it is a direct count (transactions successfully included) rather than a timing measurement.
- Block size / cumulative bytes: Very clean for the same reason: a byte count is deterministic given a fixed transaction format and demand, with no timing noise involved.
- Consensus round latency: Useful, but the noisiest of the four, since it necessarily includes operating-system scheduling and coordination overhead alongside the signal of interest (Section 6.4).

The project's central causal claim is therefore not built to rest primarily on consensus latency. Verification cost, capacity, and cumulative bytes carry the main evidentiary weight; consensus-round latency is reported and discussed as the system-level consequence of that underlying cost, corroborating the cleaner measurements rather than serving as the sole evidence for the paper's conclusions.

7. Scope Boundaries: What Will Not Be Done

The following are explicitly excluded from this project. Each exclusion exists to keep engineering effort pointed at the research question rather than at incidental technical ambition:

- A full or faithful reimplementation of Bitcoin: the project consumes btq-core's existing consensus, P2P, and cryptographic code rather than writing any of it itself, beyond the BIP-361 patch described in Section 4.5.
- Public, internet-scale, or geographically distributed deployment. The six nodes do run real P2P networking and do run on cloud VMs (Section 4.8), but on a single-region, private Virtual Cloud Network rather than across the public internet or multiple regions.
- Novel consensus protocol design (no Tendermint, HotStuff, or proof-of-work mining).
- Simulation of an actual quantum attack or cryptanalysis.
- Design of new cryptographic algorithms: Only existing, standardized schemes are used.
- Modeling of BIP-361's Phase C recovery mechanism, which has no finalized specification to model.
- Deployment beyond the six-node, free-tier VM mesh described in Section 4.8: no attempt at a larger, paid, or multi-region cluster.

- A full agent-based behavioral adoption model as a Tier 1 requirement. The core experiments use a fixed, deterministic logistic curve (Section 4.5) specifically to keep the E1-versus-E2 comparison causally clean; a richer, deadline-reactive behavioral model is an optional Tier 3 extension only (Section 8.1).
- Modification of btq-core's existing cryptographic or P2MR implementation. It is consumed as-is; the only consensus code this project writes is the BIP-361 patch.

8. Project Tiers and Timeline

8.1 Scope Hierarchy

To prevent the project from becoming hostage to its own ambition, work is organized into three explicit tiers. Completing Tier 1 alone constitutes a successful, defensible senior project; Tiers 2 and 3 strengthen it further as time allows.

Tier	Contents	Status if reached
Tier 1: Required	Six-node btq-core mesh on free-tier Oracle VMs; real classical and ML-DSA signatures via btq-core; our BIP-361 Phase A/B consensus patch; real UTXO/block/latency measurement over RPC; fixed logistic adoption curve; core E0/E1/E2 experiments; Python analysis; final comparison.	Project is complete and defensible.
Tier 2: Strengthening	Migration-speed and curve-shape sensitivity (S1, S2); cumulative overhead and phase-shock metrics.	Substantially stronger paper.
Tier 3: Stretch	System-capacity sensitivity (S3); a second ML-DSA parameter set, if available; perf-based profiling; a behavior-driven agent-based adoption model as an alternative to the fixed logistic curve, attempted only once Tier 1's core comparison is already working.	Bonus, pursued only after Tiers 1–2 are stable.

8.2 Semester 1: Build the Core System

Semester 1 is dedicated entirely to producing a complete, working end-to-end pipeline on real infrastructure: verifying VM quota and architecture, building and deploying btq-core across the 6-node private regtest mesh, writing and testing the BIP-361 patch (first on 2-3 local nodes before scaling to all six), the fixed logistic adoption trajectory, and the RPC-based metrics instrumentation. By the end of Semester 1, a complete E0 baseline run across all six real nodes, with telemetry flowing cleanly into the Python pipeline, should already exist, so that Semester 2 begins with a working system rather than an unfinished one.

8.3 Semester 2: Run the Study and Write the Paper

Semester 2 shifts focus to the full experiment matrix described in Section 5 run across the live 6-node mesh, the statistical and derived-metric analysis described in Section 6, any Tier 2 or Tier 3 extensions time permits (including the behavior-driven adoption model, attempted only after the Tier 1 comparison is confirmed working), and the writing of the final paper itself, structured around Sections 1 through 6 of this document.

8.4 Team and Division of Labor

This project is carried out by a two-person team, and the division of labor follows the two halves of the system that carry the most independent risk: the C++ consensus patch and the Python workload/telemetry layer. Both students are expected to be able to read and modify the other's core code by the end of Semester 1, so the BIP-361 patch in particular does not become a single-owner black box that the whole Semester 2 experiment matrix has to trust blindly.

- Student 1 (Consensus Lead): btq-core build and VM deployment across the mesh, the BIP-361 patch in src/validation.cpp (Phase A and Phase B), and functional testing of the patch on a small local node set before scaling to six.
- Student 2 (Systems Lead): the fixed logistic adoption trajectory generator, the RPC-based orchestration and telemetry harness across the six nodes, the Python analysis pipeline, and the statistical methodology (Section 6.3).

This split maps directly onto the module boundaries in Section 4: Student 1 owns Sections 4.2, 4.5 (enforcement half), and 4.8; Student 2 owns Sections 4.5 (trajectory half), 4.6, and 4.7.

9. Feasibility

Feasibility is reported per tier and described qualitatively rather than as a precise numeric estimate, since no statistical basis exists for a figure such as "85% likely" on a project of this kind — a specific percentage would imply a rigor the estimate does not actually have. The move to real Bitcoin nodes on free-tier VMs (Sections 3.1, 4.2, 4.8) raises the difficulty of Tier 1 relative to the original in-process design, since it now involves unfamiliar consensus-critical C++ and free-tier hardware constraints rather than a self-contained Go codebase; this is offset by the two-person team and by reusing btq-core's existing cryptography instead of writing it from scratch.

Scope	Feasibility (two-person team, two semesters, free-tier VMs)
Tier 1 (minimum viable research contribution)	Moderate-to-high, conditional on confirming free-tier VM quota and a successful btq-core build in Week 1 (Section 4.8)
Tier 1 + Tier 2	Moderate-to-high
Tier 3	Stretch / explicitly optional

This is a practical risk assessment, not a statistical estimate. The core research contribution is deliberately scoped to be achievable within two semesters by a two-person team, based on the module boundaries and scope cuts described in Sections 4, 7, and 8, and on the documented fallback in Section 4.8 (a networked version of the original in-process design) if

btq-core proves unworkable on the available free-tier hardware. The sensitivity analysis and further extensions, including the behavior-driven adoption model, are separately prioritized and pursued only once the core experiment is confirmed working, giving the project a realistic fallback position if implementation or debugging consumes more time than planned.

10. Expected Contribution and Conclusion

The finished paper is structured around a coherent chain of four questions, each answered by a specific experiment already described above.

- **What does post-quantum signing actually cost, in isolation?** Answered by the classical-versus-baseline comparison (E0).
- **What happens to the network during ordinary, gradual migration?** answered by E1.
- **Does enforcing a hard migration deadline change that picture, given the exact same underlying adoption behavior?** Answered by the controlled E1-versus-E2 comparison.
- **Is whatever difference is observed robust, or an artifact of one specific set of assumptions?** Answered by the S1–S3 sensitivity analysis.

The concluding contribution of the project is a direct, evidence-based answer to whether migration trajectory and enforcement timing materially change the peak and cumulative infrastructure cost of a post-quantum signature migration, and, if so, by how much, under a real, currently-proposed policy rather than a hypothetical one.

> *A phase-aware empirical study of the system-level consequences of post-quantum signature migration, using Bitcoin's proposed migration policy as a case study, run on real, patched Bitcoin Core nodes rather than a synthetic simulator. The node cluster is the experimental instrument; the contribution is the measurement and analysis of migration dynamics, phase enforcement, peak stress, and cumulative infrastructure cost.*